به نام خدا



دانشکده مهندسی کامپیوتر

طراحی بازی‌های کامپیوتری

استاد: جناب آقای دکتر بهروز مینایی

دستیاران: امیرحسین محمدی‌پور و محمد حقیقت

تمرین دوم

نیم‌سال اول ۱۴۰۵-۱۴۰۶

قوانین و ملاحظات تمرین

- در صورت هم‌فکری با سایر دانشجویان یا استفاده از منابعی خارج از محتوای کلاس، ذکر نام افراد و آدرس دقیق منابع الزامی است.

- فایل پروژه را در یک فایل فشرده (zip) با قالب GD_Hw2_[firstName]_[lastName]_[StudentId] نام‌گذاری کرده و در سامانه درس بارگذاری کنید.

- مسئولیت سلامت فایل آپلود شده بر عهده دانشجوست. به فایل‌های خراب (Corrupted) نمره‌ای تعلق نمی‌گیرد. حتماً پس از بارگذاری، فایل خود را دانلود و بررسی کنید.

پروژه عملی: پیاده‌سازی بازی ماشینی سه‌بعدی بی‌نهایت

در این تمرین باید یک بازی ماشینی سه‌بعدی Endless با استفاده از موتور بازی‌سازی Unity پیاده‌سازی کنید. بازیکن یک ماشین را در یک جادهٔ بی‌نهایت سه‌لاینه در دل شب از دید سوم‌شخص (Third-Person) هدایت می‌کند. ماشین با سرعت بالا در حال حرکت است و بازیکن باید با جابه‌جایی بین لاین‌ها از برخورد با ماشین‌های مانع اجتناب کند، سکه جمع کند و سوخت ماشین خود را مدیریت نماید. هدف این تمرین علاوه بر تقویت مهارت‌های فنی در توسعهٔ بازی‌های سه‌بعدی (تولید رویه‌ای محیط، بهینه‌سازی با Object Pooling و الگوهای طراحی)، توجه ویژه به جلوه‌های بصری (VFX)، نورپردازی شبانه و کیفیت هنری بازی است.

نیازمندی‌ها و ویژگی‌های بازی:

- محیط و جاده (**Procedural Road Generation**): جاده باید به‌صورت رویه‌ای (Procedural) و بی‌نهایت تولید شود. قطعات جاده باید به‌صورت پویا جلوی بازیکن ساخته شوند و پس از عبور ماشین بازیکن بازیافت گردند. جاده شامل ۳ لاین است و ماشین بازیکن با کلیدهای A/D یا فلش چپ/راست بین لاین‌ها جابه‌جا می‌شود.

- سیستم ورودی (**Input System**): تمامی ورودی‌های بازی باید منحصراً با استفاده از سیستم New Input System یونیتی (Input Actions و Action Maps) مدیریت شوند. تذکر مهم: پروژه‌هایی که با سیستم Input قدیمی پیاده‌سازی شوند، در این بخش نمره‌ای دریافت نخواهند کرد.

- گیم‌پلی و کنترل ماشین:

  — ماشین بازیکن دارای ۳ جان (Life) است که تعداد آن در رابط کاربری به بازیکن نمایش داده می‌شود.

  — برخورد با هر یک از ماشین‌های مانع باعث کسر یک جان می‌شود و باید با یک بازخورد بصری مناسب (مانند تغییر رنگ ماشین یا لرزش دوربین) همراه باشد.

  — با رسیدن تعداد جان‌ها به صفر، بازی پایان می‌یابد (Game Over).

- سیستم سوخت (**Fuel System**):

  — ماشین بازیکن دارای مخزن سوختی است که به‌مرور زمان کاهش می‌یابد.

  — در مسیر، گالن‌های سوخت (Fuel Can) قرار دارند که با جمع‌آوری آن‌ها سوخت ماشین شارژ می‌شود.

- تمام شدن سوخت به منزلهٔ پایان بازی (Game Over) است و میزان سوخت باقی‌مانده باید به‌صورت یک نوار سوخت در رابط کاربری نمایش داده شود.

- سختی پلکانی (**Progressive Difficulty**): با گذشت زمان و افزایش مسافت طی‌شده، سختی بازی باید با افزایش سرعت حرکت ماشین و تراکم ماشین‌های مانع در جاده به‌تدریج افزایش یابد.

- بهینه‌سازی با **Object Pooling**: تمام اشیای تکرارشونده در بازی (قطعات جاده، ماشین‌های مانع، سکه‌ها و گالن‌های سوخت) باید با استفاده از الگوی Object Pooling مدیریت شوند. ساخت و حذف مکرر اشیاء با Instantiate/Destroy قابل‌قبول نیست.

- طراحی شی‌گرا (**OOP Design**): بازی باید حداقل شامل دو نوع مانع (ماشین‌های ثابت و متحرک) و دو نوع آیتم جمع‌کردنی (سکه و گالن سوخت) باشد. جهت رعایت اصول شی‌گرایی، تمامی این عناصر باید از کلاس انتزاعی (abstract) مشترکی به نام `Interactable` ارث‌بری کرده و از متدهای virtual و override استفاده نمایند.

- الگوی **Singleton** برای مدیریت بازی: مدیریت وضعیت‌های بازی (Menu، Playing، Paused، GameOver) باید از طریق یک کلاس GameManager با الگوی طراحی Singleton انجام شود.

- جلوه‌های بصری، نورپردازی و آرت بازی (**Visual Art, Lighting & VFX**): علاوه بر منطق فنی بازی، جنبه‌های هنری و جذابیت بصری بازی بخش مهمی از ارزیابی این تمرین را تشکیل می‌دهد و پیاده‌سازی موارد زیر دارای امتیاز است:

  - اتمسفر شبانه (**Night Environment**): محیط بازی باید در شب جریان داشته باشد. استفاده از یک Skybox شبانه مناسب (پرستاره یا ابری)، تاریکی موضعی محیط و نورپردازی شبانه برای جاده الزامی است.

  - نورپردازی نئونی پویا (**Dynamic Neon Lighting**): ماشین بازیکن علاوه بر چراغ‌های اصلی جلو (Headlights)، باید دارای نور نئونی در زیر بدنه (Underglow Neon) باشد. شدت روشنایی (**Intensity**) و درخشش نور نئون باید متناسب با سرعت لحظه‌ای ماشین تنظیم شود (با بالا رفتن سرعت، شدت نور نئون بیشتر و درخشان‌تر شود). همچنین حضور چراغ‌های خیابانی در طول مسیر جاده توصیه می‌شود.

  - آتش‌بازی در آسمان (**Sky Fireworks**): برای پویایی و چشم‌نواز شدن فضای شب، باید یک سیستم آتش‌بازی در آسمان پیاده‌سازی شود که هر از گاهی (به‌صورت دوره‌ای یا تصادفی) پرتاب شده و در آسمان جلوهٔ رنگی زیبایی خلق کند.

- سیستم ذرات و پردازش نهایی (**Particles & Post-Processing**): استفاده از سیستم ذرات (Particle System) برای دود/آتش اگزوز ماشین، جرقه‌های ناشی از برخورد و افکت محو شدن آیتم‌ها ضروری است. همچنین استفاده از افکت Bloom (از پکیج Post-Processing) برای درخشش واقعی نورهای نئونی و انفجارهای آتش‌بازی الزامی است.

- سیستم صدا (**Audio System**): بازی باید شامل صداگذاری مناسب باشد: موسیقی پس‌زمینه، صدای موتور ماشین، صدای برخورد با موانع و صدای جمع‌آوری آیتم‌ها (سکه و سوخت).

- رابط کاربری (**UI**):
  - منوی اصلی (**Main Menu**): شامل دکمه‌های شروع بازی و خروج.
  - منوی توقف (**Pause Menu**): شامل دکمه‌های ادامه، ریستارت و بازگشت به منوی اصلی.
  - صفحه پایان بازی (**Game Over Screen**): نمایش امتیاز فعلی، رکورد بالاترین امتیاز و دکمه ریستارت.
  - **HUD** داخل بازی: نمایش زندهٔ سرعت ماشین، نوار سوخت، تعداد جان‌های باقی‌مانده، امتیاز فعلی و رکورد بالاترین امتیاز.

- ذخیره‌سازی اطلاعات (**Data Persistence**): با استفاده از سامانهٔ PlayerPrefs، رکورد بالاترین امتیاز (High Score) باید ذخیره گردد تا با خروج از بازی حفظ شود.

نحوه تحویل:

شما باید پروژه‌ی یونیتی خود را در یک مخزن (Repository) گیت (مانند گیت‌هاب یا گیت‌لب) قرار دهید. فایل ارسالی نهایی در سامانه باید یک فایل فشرده (zip) شامل موارد زیر باشد:

۱. یک فایل متنی حاوی لینک مخزن گیت پروژه (دقت کنید که حتماً از فایل `gitignore`. قرار داده شده در کنار صورت تمرین استفاده کنید تا پوشه‌های حجیم مانند Library روی گیت آپلود نشوند و تنها فایل‌های ضروری برای اجرای پروژه قرار گیرند).

۲. فایل بیلد اجرایی (Build) از بازی.

۳. یک گزارش کار کوتاه شامل توضیح ساختار پیاده‌سازی و تصاویری از خروجی نهایی بازی.